

Document 04 — OpenAI API → Project 1

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[01](#), [02](#), [03](#)

How to Read & Practice This Document

- **What:** calling OpenAI's API directly, with no extra library.
- **Why:** every library you learn later (LangChain, LangGraph) is really just a wrapper around this. To debug them later, you need to know what's underneath.
- **When:** any time you build or fix an LLM feature — library or not, this is the layer everything else sits on top of.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercises closed-book. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open. 4. Try **Project 1** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1** → **Hint 4** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud how structured output actually gets enforced under the hood. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Everything so far has been preparation. Doc01 gave you a config loader and a logger. Doc02 gave you a disciplined way to make HTTP calls. Doc03 gave you a working mental model of what happens inside a call to a model. This document puts all three to work on a real, live OpenAI call — and ends with your first real project.

First, the `openai` library is really just Doc02's HTTP layer wrapped in a nicer shape — `client.chat.completions.create(...)` sends the `messages` list (Doc03's "memory" idea, made concrete) and gets a reply back.

Second, a reply can come back all at once, or **streamed** piece by piece as it's written — the same total wait, but a very different feel for a user watching it.

Third, when your code needs to actually *use* the model's answer (not just show it to someone), asking nicely for JSON in a prompt isn't good enough — **structured outputs** forces the model's answer into a shape you define, so you get back a checked object or a clear failure, never a guess.

Fourth, the library's errors map directly onto Doc02's retry rules — `AuthenticationError` and `BadRequestError` are permanent (fix the request, don't retry), `RateLimitError` and `APITimeoutError` are temporary (back off and retry) — so everything you learned in Doc02 about *which* failures to retry applies here without any new rules to learn.

Fifth, there are two APIs for this — Chat Completions (the steady, foundational one you're learning here) and the newer Responses API (which manages conversation history for you, better suited to agent-heavy work later). You start with Chat Completions because it's what almost everything else, including most of LangChain, is actually built on underneath.

That's the story: this document turns three documents' worth of preparation into one real, working call to a real model — and the **Build Task** (Project 1) asks you to put a full small app around it: a chatbot with real memory, streaming replies, structured output, and graceful handling of the failures Doc02 and Doc03 already taught you to expect.

Core Concepts (read this first — everything you need is here)

Topics on this page: [The client, and the list of messages](#) · [Streaming vs. waiting](#) · [Structured output](#) · [Error types](#) · [Chat Completions vs. Responses API](#)

The client, and the list of messages

The `openai` library wraps the raw HTTP calls from Doc02 into an easier-to-use client: `client = OpenAI(api_key=...)`, then `client.chat.completions.create(model=..., messages=[...])`. The `messages` list is the whole conversation, written as a sequence of entries, each with a role: `{"role": "system", "content": "..."}` sets the model's behavior once, near the top. `{"role": "user", ...}` and `{"role": "assistant", ...}` go back and forth as the conversation continues. **Why the system role is separate from the user role:** it's a way to give the model standing instructions that carry more weight than a regular user message, and keeping it separate makes it easy to change the model's behavior without touching your conversation logic. **How "memory" actually gets built** (tying back to Doc03): every new call resends the *whole* growing `messages` list — there's no `add_message()` call on some server-side conversation object. You're the one appending to the list, and sending the whole thing again each time.

Streaming vs. waiting for the full reply

A normal (non-streaming) call waits until the model has finished writing its whole answer, then gives it to you all at once. A streaming call (`stream=True`) instead sends back small pieces as they're written, so you can show the words as they appear — the difference between a chatbot that seems

to "type" live, versus one that pauses, then dumps a whole paragraph at once. **Why this matters beyond just looks:** for a long answer, streaming makes it *feel* faster (the first word shows up quickly) even though the total time to finish is the same. It also lets you start processing or logging part of the answer before the whole thing is done. **When not to bother:** a structured-output call that you're going to read as one complete piece of data usually doesn't need streaming — you need the whole thing before it's useful anyway.

Structured output: how "give me JSON" is actually guaranteed

Just asking the model in a prompt to "answer in JSON" is a request, not a guarantee — it can still write broken JSON, or drift away from the shape you wanted. **Structured outputs** is a stronger method: you give it a shape to follow (usually through a Pydantic model), and the API limits what the model can generate so the output *has* to match that shape — not just "asked nicely," but actually restricted while it's being written. **Why this matters:** it turns "I hope the model got the format right" into "the library hands me back a checked Python object, or fails clearly" — which removes a whole category of fragile, manual JSON-parsing bugs. **When to use it:** any time code downstream needs to actually use the model's answer (pulling out data, sorting into categories, deciding what tool to call) rather than just showing it to a person.

Error types, and which Doc02 rules apply to each

The library raises clear, named errors that match the HTTP categories from Doc02:

`AuthenticationError` (bad or missing key — this is permanent, never retry), `RateLimitError` (too many requests — this is temporary, retry with backoff), `APITimeoutError` (temporary, retry), `BadRequestError` (your request itself was wrong — like going over the context limit — permanent, fix the request instead of retrying). **Why named errors matter here specifically:** catching each one separately lets your code respond correctly to each kind of failure — showing a config problem right away, versus quietly retrying a rate limit — instead of one big `except Exception` that treats a typo'd API key the same as a brief network hiccup.

Chat Completions vs. the newer Responses API

Chat Completions is the original, steady API — you send the full message history every time, as explained above. The newer **Responses API** is OpenAI's current recommendation for building agent-style, tool-heavy apps: it can keep track of conversation history on OpenAI's own servers instead of you resending it every time, and it's built with tool use more in mind from the start. **Why you're learning Chat Completions first anyway:** it's still what most existing production code, most of LangChain's model wrapper, and most tutorials use underneath. Understanding it means understanding the foundation everything else sits on. **When to reach for the Responses API instead:** new agent-style projects where you'd rather have the platform manage conversation history for you — you'll come back to this once you're comfortable with the basics here.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [OpenAI API reference](#) — the source of truth. Bookmark it, you'll come back often.
- [OpenAI — Text generation / Chat Completions guide](#) — the main call you'll use most.
- [OpenAI — Function calling guide](#) — read now to see the shape of it; full use starts in Doc06.
- [OpenAI — Structured outputs guide](#) — checked, guaranteed-shape answers.
- [openai-python SDK \(GitHub\)](#) — the README and `examples/` folder.
- [openai-cookbook \(GitHub\)](#) — real worked examples; skim the titles, read 2-3 that look useful.
- [OpenAI — Responses API reference](#) — the newer API.

Practice Exercises

Setup for this document's practice code: work inside `04_openai_api/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install openai python-dotenv pydantic` .

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py` , with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python conversation_memory_practice.py` .

For this document, save your practice code as: - **Basic** (your first real API call) is its own topic — save it as `chat_api_basics_practice.py` . - **Intermediate** (build real memory) is its own topic — save it as `conversation_memory_practice.py` . - **Real-world** (make it feel alive with streaming) is its own topic — save it as `streaming_practice.py` . - **Edge cases** (what happens when you go over the limit) is its own topic — save it as `context_limit_practice.py` . - **Failure** (a bad key, and a cost comparison) is its own topic — save it as `auth_and_cost_practice.py` .

None of this document's exercises share the same underlying topic, so each one gets its own file this time — but the rule still stands: check every document for real overlaps first.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — your first real API call

- **What:** one call to the model, print the reply, then change the system prompt and watch the answer change.
- **Why:** this is the single call every other document and project builds on — get it working with your own eyes before adding anything else.
- **When you'll hit this for real:** literally Step 1 of Project 1.
- **How to code it:** `client = OpenAI()` , then `client.chat.completions.create(model="gpt-4o-mini", messages=[{"role": "system", "content": "..."}, {"role": "user", "content": "..."}])` . Print `response.choices[0].message.content` . Change the system prompt and run it again.
- **Save as:** `chat_api_basics_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — build real memory

- **What:** a loop that sends multiple turns, adding each one to a growing message list.
- **Why:** this is what "memory" means for an LLM — there's no other kind. Building it yourself once means you'll never be confused by it again.
- **When you'll hit this for real:** Project 1 Step 2, and every chat feature you'll ever build.
- **How to code it:** start with `history = [{"role": "system", "content": "..."}]` . In a loop: `history.append({"role": "user", "content": input()})` , call the API with `history` , append the reply too, and print it.
- **Save as:** `conversation_memory_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — make it feel alive with streaming

- **What:** switch the same call to `stream=True` and print words as they arrive instead of waiting for the whole reply.
- **Why:** this is the difference between a chatbot that feels instant and one that feels like it's stalling — a real, user-facing quality difference, not just a technical detail.
- **When you'll hit this for real:** any chat UI where the user is watching and waiting — which is most of them.
- **How to code it:** add `stream=True` to your call, then `for chunk in response:` `print(chunk.choices[0].delta.content or "", end="")` .
- **Save as:** `streaming_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — what actually happens when you go over the limit

- **What:** force a too-long input on purpose and see the exact error OpenAI gives you.
- **Why:** reading about "context limit errors" is not the same as seeing the real exception type and message once, with your own eyes.
- **When you'll hit this for real:** a long conversation, or someone pasting a huge document into your chat feature.
- **How to code it:** build a message with tens of thousands of repeated words, send it, and wrap the call in `try/except` to see and print the exact error type you get back.
- **Save as:** `context_limit_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — a bad key, and a cost comparison

- **What:** use a wrong API key on purpose and catch `AuthenticationError` specifically. Then, separately, compare speed and cost between a short system prompt and a long one, for the same task.
- **Why:** a bare `except` here would hide a config problem from you and your users forever — catching the specific error is what makes it fixable. The cost comparison builds the same intuition Doc03 started, now backed by a real, running script.
- **When you'll hit this for real:** a `.env` typo that ships to production, or a system prompt that's grown 5x longer than it needs to be without anyone noticing the cost.
- **How to code it:** temporarily set `OPENAI_API_KEY` to a fake string, catch `openai.AuthenticationError` specifically, print a clean message. Then run the same task twice — once with a 1-sentence system prompt, once with a 20-sentence one — and compare the token counts OpenAI reports back in the response.
- **Save as:** `auth_and_cost_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Project 1: Beginner LLM App

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a terminal chatbot with real conversation memory, and one structured-output feature.

Requirements:

- A multi-turn conversation loop in the terminal, using the config/logger from Doc01.
- Uses Doc02's `http_client` style where it makes sense (or the library's own retries — pick one, and be able to explain why).
- At least one mode that gives back a checked, Pydantic-shaped answer (like "pull these fields out of what I just said").
- Handles gracefully: a bad key, a rate limit, a too-long conversation, a bad structured-output reply.

Inputs: whatever the user types into the terminal.

Outputs: streamed replies in the terminal; a checked, structured object for the structured-output feature.

Constraints: no secrets in code; full type hints; must not crash on any of the 4 problems below — it should handle each one gracefully, with a clear message.

Suggested files:

```
project_1_supportdesk_chat_and_triage/
├─ main.py
├─ chat_client.py
├─ schemas.py
├─ config.py           (reused from 01_python_foundations)
└─ test_chat_client.py
```

Functions/Components to build:

- `chat_client.py` → `create_client()` , `send_message(history, user_input) -> str` , `stream_message(history, user_input)`
- `schemas.py` → at least one Pydantic model for the structured-output feature
- `main.py` → the terminal loop, connecting everything together

Expected Behavior

- The conversation remembers earlier messages within one session (not across restarts — that's Doc09's job).
- A bad key → a clear error message, the program exits cleanly, no scary stack trace shown to the user.
- A too-long conversation → caught and reported, not an unhandled crash.
- A structured-output request → either a valid, checked object, or a clear "couldn't understand that" message — never a silent wrong answer.

Test Cases

Scenario	Expected
A normal 3-turn conversation	Each reply reflects the earlier turns
A wrong API key	<code>AuthenticationError</code> caught, clean exit

Scenario	Expected
Input designed to go over the context limit	Handled, user is told, no crash
Structured-output request with a clear input	A valid Pydantic object comes back
Structured-output request with an unclear input	Either a best-effort object, or a clear failure — your choice, and write down why

Break-It / Debug Preview

- A message list that keeps growing until it quietly goes over the context limit in the middle of a conversation.
- Hitting a rate limit in the middle of a streamed reply. Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Chat Completions vs. Responses API · streaming vs. non-streaming trade-offs · how structured output is actually enforced · why "memory" is just resent history.

☐ You Can Now Build Project 1

Docs 01-04 are everything Project 1 needs. Go to [project_1_supportdesk_chat_and_triage/](#) and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 1 runs start to finish, handles at least 3 of the problems above gracefully, and you can explain every line without help. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** through **Hint 4**. Ask for the full solution only if you say "**Show me the solution.**"